Levenshtein Distance

AlgoExpert · Medium · Dynamic Programming

The problem

- Minimum single-char inserts/deletes/substitutions turning a into b.

Intuition

- Dynamic programming over prefixes: each cell is the cheapest edit between two prefixes.

Approach

- 1. Row zero and column zero are insert or delete costs
- 2. Equal characters copy the diagonal
- 3. Otherwise one plus the min of insert, delete, replace
- 4. Two rows suffice

Java solution

```java
public class LevenshteinDistance {
    public static int levenshteinDistance(String a, String b) {
        if (a.length() < b.length()) { String t = a; a = b; b = t; }
        int[] prev = new int[b.length() + 1], cur = new int[b.length() + 1];
        for (int j = 0; j <= b.length(); j++) prev[j] = j;
        for (int i = 1; i <= a.length(); i++) {
            cur[0] = i;
            for (int j = 1; j <= b.length(); j++)
                cur[j] = a.charAt(i - 1) == b.charAt(j - 1) ? prev[j - 1]
                        : 1 + Math.min(prev[j - 1], Math.min(prev[j], cur[j - 1]));
            int[] t = prev; prev = cur; cur = t;
        }
        return prev[b.length()];
    }
}
```

Complexity

- Time: O(n*m)
- Space: O(min(n, m))

Recap

- Dynamic programming over prefixes: each cell is the cheapest edit between two prefixes.
- Complexity: O(n*m)